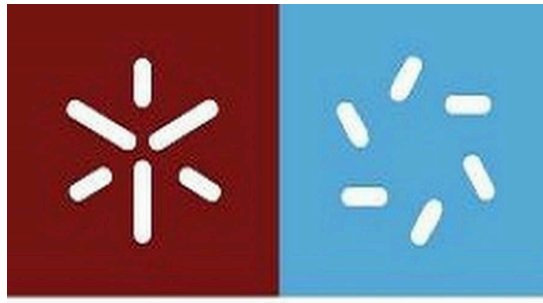




Universidade do Minho
Escola de Ciências

Computação Gráfica

2024/2025

Trabalho Prático

Fase 1 - Primitivas Gráficas

Grupo 12

Afonso Martins n°A102940

Luis Silva n° A105530

Pedro Gomes n° A102929

Ricardo Vilaça n°A102879

Índice:

1) Introdução	3
2) Gerador	3
2.1) Plane	4
2.2) Box	5 – 6
2.3) Cone	6 – 8
2.4) Esfera	8 – 9
3) Engine	9
3.1) Escolha da Biblioteca	10
4) Conclusão	11

1) Introdução:

Nesta primeira fase, ao grupo foram atribuídas as seguintes tarefas:

- **Geração de ficheiros** contendo os modelos de informação, onde apenas os vértices do modelo devem ser gerados.
- **Desenvolvimento de uma *engine*** capaz de ler o ficheiro de configuração (*configuration file*), escrito em XML, e exibir os modelos construídos.

Estas etapas são fundamentais para estabelecer a base do projeto, garantindo a correta representação e visualização dos modelos gráficos.

Para a implementação das tais tarefas, utilizamos a linguagem C++ e a ferramenta OpenGL, que as mesmas foram lecionadas nas aulas.

2) Gerador:

O nosso gerador, foi construído no ficheiro *gerador.cpp* com a capacidade de criar representações de primitivas geométricas e guarda a informação em outro ficheiro.

Como Funciona?

O programa recebe argumentos do tipo:

```
./programa < primitiva > < parâmetros > < ficheiro_saida >
```

Por exemplo:

```
./programa plane 10 5 output.txt
```

- < primitiva > : Tipo de primitiva que pretendemos criar/gerar(plano, box, cone ou esfera);
- < parâmetros > : Parâmetros necessários para construir cada primitiva:
 - esfera:
 - raio
 - slices
 - stacks
 - plano:
 - comprimento
 - divisões
 - caixa:
 - comprimento
 - divisões
 - cone:
 - raio
 - altura
 - slices
 - stacks
- < ficheiro_saida > : Ficheiro onde ficará guardado o modelo;

2.1) Plane:

O plano vai estar a contido no plano XZ, com o seu centro na origem (0,0,0). A sua construção é feita a partir de uma matriz de subdivisões de dimensão $d \times d$ (d = divisões), cada célula da matriz vai representar um quadrado. Este é dividido em dois triângulos, onde a sua diagonal forma a hipotenusa do triângulo.

Para a construção dos pontos iniciais do plano realizamos os seguintes cálculos:

Sendo:

$$\text{step} = \text{length}/\text{division}$$

$$x = (\text{length}/2) - j * \text{step}$$

$$y = 0$$

$$z = (\text{length}/2) - i * \text{step}$$

Com isto os quatro pontos iniciais para o plano vão ser constituídos da seguinte forma:

$$\begin{aligned} p_0 &= (x_{i,j}, y, z_{i,j}) \\ p_1 &= (x_{i,j}, y, z_{i+1,j}) \\ p_2 &= (x_{i,j+1}, y, z_{i,j+1}) \\ p_3 &= (x_{i,j+1}, y, z_{i+1,j+1}) \end{aligned}$$

Com isto temos os triângulos formados, o primeiro triângulo é composto pelas ligações de:

$$p_0 \rightarrow p_1 \rightarrow p_2$$

E o segundo triângulo composto pelas ligações de:

$$p_1 \rightarrow p_2 \rightarrow p_3$$

Através do **GL_TRIANGLE_STRIP**(modo de renderização de triângulos utilizado no OpenGL, que permite desenhar uma sequência de triângulos de forma bastante eficiente), conseguimos automatizar a criação de triângulos. Assim como podemos ver na figura abaixo.

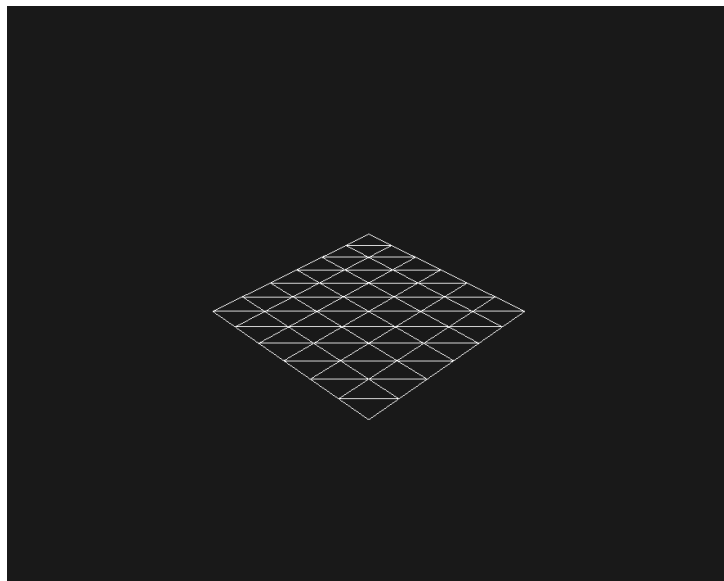


Figure 1: Plano com comprimento 2 e 6 sub-divisões.

2.2) Box:

A próxima figura a ser gerada é a “box” ou caixa, para isto a mesma é dividida em pequenas células que utilizam o **GL_TRIANGLE_STRIP**. Para as faces da caixa, a mesma é subdividida em $d \times d$, onde cada célula é composta por dois triângulos.

Assim como foi feito o plano utilizamos fórmulas para a construção da caixa, sendo:

half = length / 2.0f -> Definição do meio do comprimento da caixa.

divisions -> Número de subdivisões de cada face.

step = length/divisions -> Tamanho de cada subdivisão.

Para a construção da caixa sabemos que a mesma é composta por seis faces, agora dá-se ao processo de construção de cada uma das faces:

Face Inferior(-Y):

Os primeiros pontos a serem gerados são no plano **XZ** com o valor de $y = -(\text{length}/2)$ e gera *divisions* ao longo do **eixo Z**, para a criação dos vértices dá-se os seguintes valores a x e a z :

$$x = (\text{length}/2) - j * \text{step}$$

$$y = -(\text{length}/2)$$

$$z = (\text{length}/2) - i * \text{step}$$

Face Superior(+Y):

Pontos gerados no plano **XZ** com o valor de $y = (\text{length}/2)$, semelhante à face anterior:

$$x = (\text{length}/2) - j * \text{step}$$

$$y = (\text{length}/2)$$

$$z = (\text{length}/2) - i * \text{step}$$

Face Lateral(-X):

Pontos gerados no plano **YZ** com o valor de $x = -(\text{length}/2)$ e gera *divisions* ao longo do **eixo do Y**, para criação dos dos vertices da mesmas dá-se os valores a y e a z :

$$x = -(\text{length}/2)$$

$$y = (\text{length}/2) - j * \text{step}$$

$$z = (\text{length}/2) - i * \text{step}$$

Face Lateral(+X):

Pontos também gerados no plano **YZ** mas com o valor de $x = (\text{length}/2)$, com os valores para y e z :

$$\begin{aligned}x &= (\text{length}/2) \\y &= (\text{length}/2) - j * \text{step} \\z &= (\text{length}/2) - i * \text{step}\end{aligned}$$

Face Lateral(-Z):

Para face, são criados também os pontos x , y , z na face **XY** com os valores de:

$$\begin{aligned}x &= (\text{length}/2) - j * \text{step} \\y &= (\text{length}/2) - i * \text{step} \\z &= -(\text{length}/2)\end{aligned}$$

Face Lateral(+Z):

Para finalizar criamos os pontos finais para criar a caixa, com os valores de:

$$\begin{aligned}x &= (\text{length}/2) - j * \text{step} \\y &= (\text{length}/2) - i * \text{step} \\z &= (\text{length}/2)\end{aligned}$$

Com isto tudo, podemos compilar o gerador com os argumentos para gerar a caixa e obtemos a o resultado demonstrado na figura abaixo:

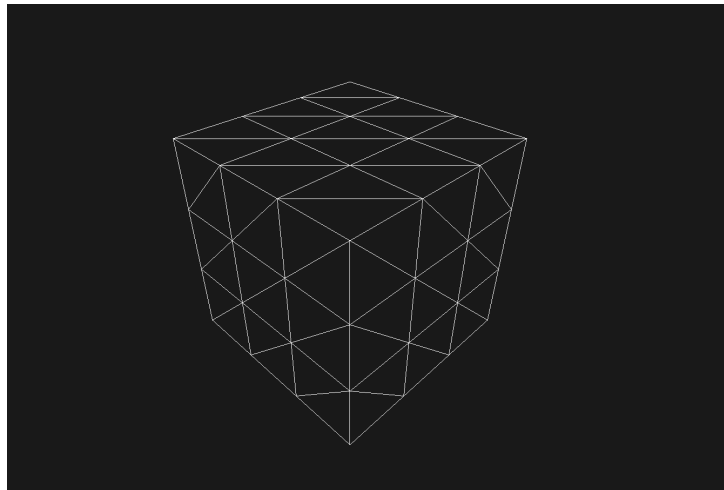


Figure 2: Caixa com comprimento 3 e 3 sub-divisões.

2.3) Cone:

A próxima figura a ser gerada é o cone. O cone é composto por uma base, gerada com **GL_TRIANGLE_FAN**, e um corpo. O corpo do cone é composto por n faces, sendo $n = \text{slices}$, cada

uma gerada por um **GL_TRIANGLE_STRIP**. Cada face está ainda dividida em k partes, sendo k = stacks.

Assim como foi feito o plano utilizamos fórmulas para a construção do cone, sendo:

radius -> Raio do cone.

height -> Altura do cone.

slices -> Número de slices do cone.

stacks -> Número de stacks do cone.

angleStep = 360/slices -> Tamanho do incremento do ângulo de uma face para a próxima;

heightStep = height/stacks -> Tamanho do incremento da altura de uma stack para a próxima;

ratio = radius/height -> rácio entre o raio e a altura do cone;

Base assente no plano XZ:

Para gerar a base temos que **y = 0** para todos os pontos da base, enquanto que para cada triângulo que compõe a base são calculados os pontos **x**, **z**, **nextX** E **nextZ** com as fórmulas:

$$angle = i * angleStep * M_PI / 180$$

$$nextAngle = (i + 1) * angleStep * M_PI / 180$$

$$y = 0$$

$$x = radius * \sin(angle)$$

$$z = radius * \cos(angle)$$

$$nextX = radius * \sin(nextAngle)$$

$$nextZ = radius * \cos(nextAngle)$$

Corpo do cone:

O corpo do cone é construído face a face. Cada face começa com o ponto mais alto do cone, ou seja **x = 0**, **z = 0** e **y = height**, e depois são calculados os pontos **x**, **y** e **z** de cima para baixo, sendo que a cada dois pontos é desenhada uma stack:

$$angle = i * angleStep * M_PI / 180$$

$$nextAngle = (i + 1) * angleStep * M_PI / 180$$

$$stackHeight = height - heightStep * (j + 1)$$

$$stackRadius = (height - stackHeight) * ratio$$

$$x = stackRadius * \sin(angle)$$

$$y = stackHeight$$

$$z = stackRadius * \cos(angle)$$

$$nextX = stackRadius * \sin(nextAngle)$$

$$nextZ = stackRadius * \cos(nextAngle)$$

Podemos agora usar o engine para renderizar o cone definido por estes parâmetros.

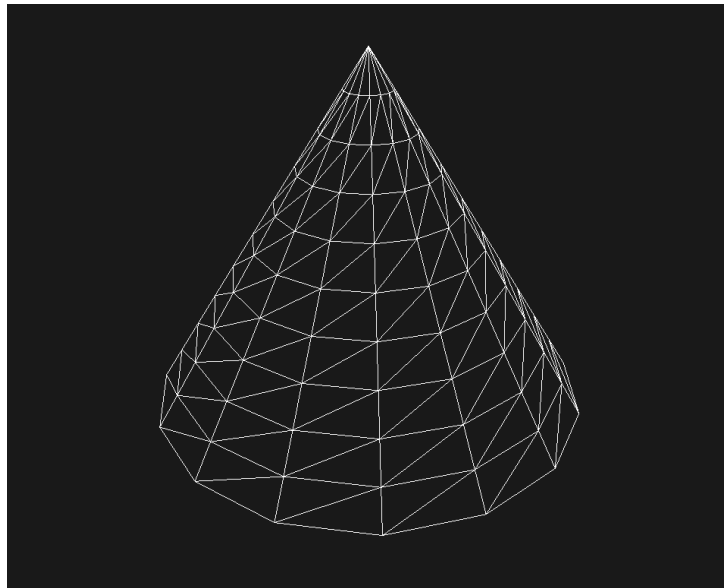


Figure 3: Cone com raio 2, altura 3, 15 slices e 10 stacks.

2.4) Esfera:

A última figura que pode ser gerada é a esfera. A esfera está dividida em n slices verticais, sendo cada uma destas composta por k sub-divisões ou stacks. Para a geração da esfera utiliza-se um **GL_TRIANGLE_STRIP** para cada slice, pelo que é similar ao processo de criar uma face do cone como descrito em cima.

Abaixo estão as fórmulas usadas para calcular o x , y e z :

radius -> Raio da esfera.

slices -> Número de slices do cone.

stacks -> Número de stacks do cone.

angleStepH = $360/\text{slices}$ -> Tamanho do incremento do ângulo na horizontal de uma slice para a próxima;

angleStepV = $180/\text{slices}$ -> Tamanho do incremento do ângulo na vertical de uma stack para a próxima;

Slice da esfera:

Cada slice da esfera começa com o ponto mais alto da esfera, ou seja $x = 0$, $z = 0$ e $y = \text{radius}$, e depois são calculados os pontos x , y e z de cima para baixo, sendo que a cada dois pontos é desenhada uma stack, terminando no ponto mais baixo da esfera, ou seja $x = 0$, $z = 0$ e $y = -\text{radius}$:

$$\text{angleV} = (j + 1) * \text{angleStep} * M_PI / 180$$

$$\text{angleH} = i * \text{angleStep} * M_PI / 180$$

$$\text{nextAngleH} = (i + 1) * \text{angleStep} * M_PI / 180$$

$$x = \text{radius} * \sin(\text{angleH}) * \sin(\text{angleV})$$

$$y = \text{radius} * \cos(\text{angleV})$$

$$z = \text{radius} * \cos(\text{angleH}) * \sin(\text{angleV})$$

$$\text{nextX} = \text{radius} * \sin(\text{nextAngleH}) * \sin(\text{angleV})$$

$$\text{nextZ} = \text{radius} * \cos(\text{nextAngleH}) * \sin(\text{angleV})$$

Podemos agora usar o engine para renderizar a esfera definida por estes parâmetros

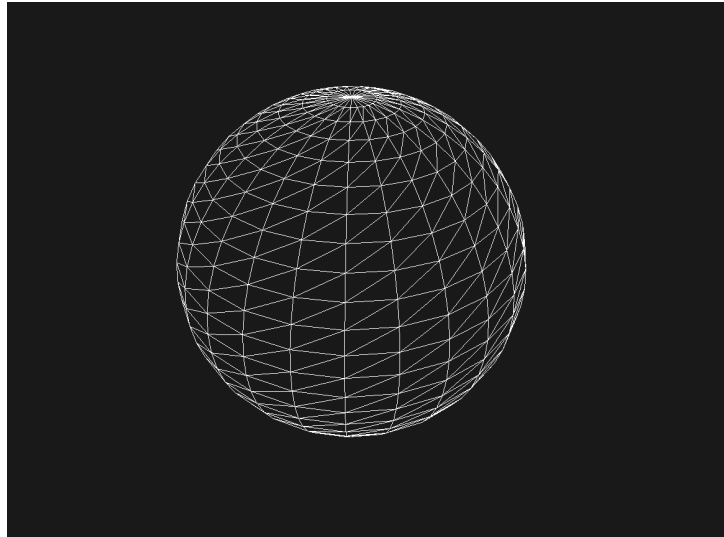


Figure 4: Esfera com raio 2, 25 slices e 25 stacks.

3) Engine:

A engine começa por carregar um ficheiro XML de configuração, que define parâmetros essenciais como a resolução da janela, as definições da câmara e os modelos a serem renderizados no ambiente 3D. Os dados da câmara incluem a posição, a direção para onde está a olhar, o vetor up e os parâmetros de projeção, como campo de visão e planos de recorte. Cada modelo 3D é carregado a partir de um ficheiro externo, onde os vértices são lidos e armazenados numa estrutura específica, que também guarda a organização das faces e a sua disposição.

O motor gráfico utiliza a biblioteca OpenGL para inicializar o contexto gráfico, ativando funcionalidades como teste de profundidade e culling de faces para otimizar a renderização. O cenário é renderizado na função display, onde a matriz de projeção é configurada e a câmara é posicionada utilizando gluLookAt. Os eixos coordenados são desenhados para referência visual e, em seguida, os modelos são percorridos e desenhados conforme o seu tipo, utilizando primitivas gráficas apropriadas, como GL_TRIANGLE_STRIP e GL_TRIANGLE_FAN.

Para garantir um loop contínuo e responsivo, a GLUT é utilizada para gerir a janela e atualizar o cenário sempre que necessário, através de glutMainLoop, permitindo assim a interação com a aplicação e a visualização dos modelos carregados. No próximo ponto, vamos discutir sobre a biblioteca a utilizar.

3.1) Escolha da Biblioteca:

Para o desenvolvimento da nossa engine, optámos por utilizar a biblioteca PugiXML para a análise e manipulação de ficheiros XML. Esta decisão foi tomada após uma análise aprofundada de três bibliotecas populares: RapidXML, TinyXML2 e PugiXML. A tabela abaixo apresenta os critérios que considerámos para fundamentar a nossa escolha.

Critério	RapidXML	TinyXML2	PugiXML
Desempenho	Extremamente rápido (parsing in-place)	Moderado (parser tradicional)	Muito rápido (quase ao nível do RapidXML)
Consumo de Memória	Baixo, mas modifica a string original	Moderado	Baixo, sem modificar a string original
Leitura de XML	✓	✓	✓
Escrita de XML	Limitada	✓	✓
API (Interface)	Complexa	Simple	Simple, com mais funcionalidades
Suporte a XPath	✗	✗	✓
Instalação	Apenas cabeçalho (rapidxml.hpp)	Biblioteca compilada (.lib / .so)	Apenas cabeçalho (pugixml.hpp)
Validação de XML	✗	✗	✗
Facilidade na correção dos Erros	Difícil	Relativamente fácil	Relativamente fácil
Compilação	Muito rápida	Moderada	Rápida
Plataformas Suportadas	C++ padrão	C++ padrão	C++ padrão

Em resumo, escolhemos a biblioteca PugiXML por oferecer o melhor equilíbrio entre simplicidade, desempenho e funcionalidades. Trata-se de uma solução eficiente e intuitiva, capaz de satisfazer os requisitos do nosso projeto sem comprometer a performance ou a facilidade de integração.

4) Conclusão:

Durante esta primeira etapa do trabalho, aplicámos na prática os conceitos abordados nas aulas , aprofundando a nossa compreensão da matéria.

Apesar dos desafios encontrados ao longo do processo, conseguimos alcançar os objetivos definidos para esta fase, concretizando a implementação do Generator e do Engine como foi pedido.

Com esta base sólida, estamos confiantes e preparados para prosseguir com as próximas fases do desenvolvimento.